
Khalifa University of Science and Technology

Computer & Information Engineering Department

Computer Networks Laboratory — CCEN 356

Design, Implementation, and Analysis of HTTP/HTTPS Performance and Visibility

Using Cisco Devices and Python Automation

Project Final Report

Project Team — Section B3, Group 1

Abd Alrahman Ismaik	100064692@ku.ac.ae
Rashid Alzarooni	100061079@ku.ac.ae
Tarek Alhafez	100064737@ku.ac.ae

Lab Instructor: Eng. Herminio Jamin

Instructor Email: herminio.vendiola@ku.ac.ae

Submission Date: April 28, 2026

Abstract

This project presents the design and implementation of a multi-segment network infrastructure for the comparative analysis of HTTP and HTTPS traffic performance, with a focus on encryption overhead, latency behaviour, and the impact of Quality of Service (QoS) policy enforcement.

The experimental topology comprises two Cisco 2901 routers interconnected over a point-to-point WAN link, a Layer 2 Cisco switch, two client workstations, and a dedicated web server. Devices were configured with static IP addressing and inter-network routing was established via static routes. The web server was provisioned with both HTTP and HTTPS services, the latter utilising a self-signed TLS certificate. Traffic was captured and analysed at the packet level using Python-based Scapy scripts, enabling protocol inspection, packet logging, and correlation with application-layer performance measurements.

Automation was central to the project methodology. Python scripts leveraging the Netmiko library performed remote SSH-based collection of router state, including interface status, routing tables, and Access Control List (ACL) and QoS configurations. Complementary scripts measured per-request response times, computed aggregate latency statistics, and produced publication-quality charts using Matplotlib. A Flask-based real-time monitoring dashboard was developed to consolidate performance metrics, endpoint availability, and TLS overhead visualisations into a unified interface.

Empirical results confirm that HTTP exhibits lower average latency than HTTPS owing to the absence of TLS handshake and symmetric-encryption processing. Conversely, HTTPS provides confidentiality and integrity guarantees critical to contemporary networked applications. QoS enforcement on Router 1 demonstrably reduced latency variance and improved traffic stability under load. This project deepened the team's understanding of application-layer protocol behaviour, router configuration, and the integration of network automation into operational workflows.

Keywords: HTTP, HTTPS, TLS, QoS, Network Performance, Cisco IOS, Python Automation, Netmiko, Scapy, Latency Analysis.

Contents

Abstract	1
1 Project Overview and Objectives	4
1.1 Background and Motivation	4
1.2 System Overview	4
1.3 Project Objectives	5
2 Project Implementation	6
2.1 Network Topology Design	6
2.2 IP Addressing Scheme	7
2.3 Device Configuration	8
2.3.1 Router Interface Configuration	8
2.3.2 Static Routing	8
2.3.3 Access Control Lists (ACL)	9
2.3.4 Quality of Service (QoS)	10
2.3.5 Switch Configuration	10
2.4 Client Workstation Configuration	11
2.5 Server Configuration	11
2.5.1 Network and Web Server Setup	11
2.5.2 TLS Certificate Installation	12
2.6 System Pipeline	13
2.7 Python Automation Implementation	13
2.7.1 SSH Automation with Netmiko	13
2.7.2 HTTP/HTTPS Performance Measurement	14
2.7.3 Packet Capture with Scapy	14
2.7.4 Visualisation with Matplotlib	14
2.8 Real-Time Monitoring Dashboard	16
2.9 Summary of Implementation	18
2.10 Machine Learning (Optional Advanced Analysis)	18
3 Project Evaluation	19
3.1 Test Methodology	19
3.2 Connectivity Testing	19
3.3 Routing Verification	20
3.4 HTTP and HTTPS Testing	20

3.5	Scapy Traffic Analysis	21
3.6	Performance Analysis (HTTP vs HTTPS)	22
3.7	QoS Impact Evaluation	23
3.8	Python Automation Results	24
3.9	Dashboard Analysis	26
4	Discussion and Conclusion	28
4.1	Key Learnings	28
4.2	Challenges Encountered	29
4.3	Broader Impact and Real-World Relevance	29
4.4	Conclusion	30
	Team Contribution	31

Chapter 1

Project Overview and Objectives

1.1 Background and Motivation

The proliferation of web-based services has made HTTP and HTTPS the dominant transport mechanisms for application-layer communication. HTTP (HyperText Transfer Protocol) is a stateless, text-based protocol that enables straightforward data exchange between clients and servers; however, its plaintext nature leaves transmitted content vulnerable to eavesdropping and tampering. HTTPS addresses these deficiencies by encapsulating HTTP within a Transport Layer Security (TLS) session, providing confidentiality, integrity, and server authentication at the cost of additional computational and temporal overhead.

Understanding the performance implications of TLS in a realistic network environment is of considerable practical importance. Network engineers and application architects must weigh security requirements against latency budgets, particularly in latency-sensitive use cases such as interactive web applications or embedded IoT devices. Quality of Service mechanisms further complicate this picture by introducing traffic prioritisation policies that can affect the observable behaviour of both protocols.

This project was motivated by the need to quantify these trade-offs in a controlled, instrumented environment using industry-standard Cisco networking hardware, contemporary packet-analysis tools, and modern Python-based automation frameworks.

1.2 System Overview

The implemented system consists of a small-scale enterprise-like network in which two geographically separated local-area networks are connected via a simulated WAN link. The client-side LAN contains two workstations that generate HTTP and HTTPS traffic toward a web server hosted on the server-side LAN. The two LANs are interconnected through a pair of Cisco 2901 routers.

Traffic monitoring occurs at multiple points in the topology. Scapy-based packet captures provide packet-level visibility, while Python scripts instrument higher-level performance metrics. A real-time dashboard aggregates these metrics and presents them in a browser-accessible interface, enabling continuous observation of protocol behaviour.

1.3 Project Objectives

The following objectives guided the design and execution of the project:

- O1.** Design and implement a functional network topology using Cisco 2901 routers and a Cisco Layer 2 switch.
- O2.** Assign static IP addressing and configure inter-network routing to establish end-to-end connectivity between all devices.
- O3.** Deploy HTTP and HTTPS services on the server and validate accessibility from both client workstations.
- O4.** Configure ACLs and QoS policies on Router 1 to filter and prioritise web traffic.
- O5.** Capture and analyse HTTP and HTTPS traffic at the packet level using Scapy-based Python scripts.
- O6.** Develop Python automation scripts to collect router state via SSH (Netmiko) and measure application-layer performance metrics.
- O7.** Quantitatively compare HTTP and HTTPS performance with respect to average latency, response time, and connection reliability.
- O8.** Evaluate the effect of QoS enforcement on traffic latency variance and stability.
- O9.** Visualise all collected data using Matplotlib charts and a live Flask dashboard.

Chapter 2

Project Implementation

2.1 Network Topology Design

The network topology separates the client-side and server-side environments into distinct /24 subnets interconnected through a /30 point-to-point WAN segment. This architecture mirrors a simplified enterprise branch-office model and provides a clear demarcation for routing, ACL enforcement, and QoS policy application.

The topology comprises the following devices:

- **Router 1 (R1):** Connects the client LAN to the WAN link; hosts ACL and QoS policies.
- **Router 2 (R2):** Connects the server LAN to the WAN link; provides the server-facing gateway.
- **Switch (SW1):** Layer 2 device forwarding frames between both client PCs and Router 1.
- **PC1 and PC2:** Client workstations that generate HTTP and HTTPS requests and execute Python measurement scripts.
- **Server PC:** Hosts Apache/Nginx with both HTTP (port 80) and HTTPS (port 443) virtual hosts, provisioned with a self-signed TLS certificate.

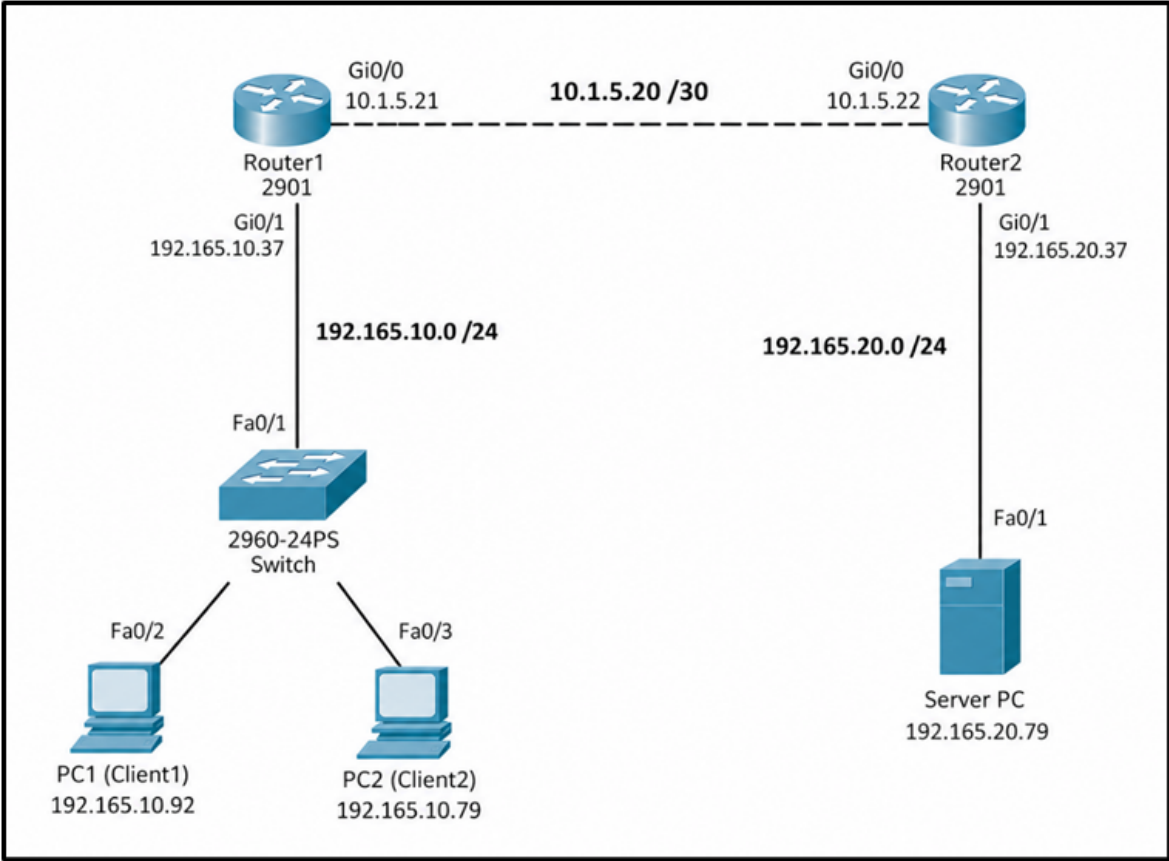


Figure 2.1. Network Topology Diagram — physical/logical layout of routers, switch, clients, and server

2.2 IP Addressing Scheme

Three distinct networks were employed in accordance with the topology design. The /30 prefix for the WAN link was selected for address-space efficiency, providing exactly two usable host addresses. The /24 prefixes for the LANs accommodate future expansion while keeping subnetting simple.

Table 2.1. IP Addressing Scheme

Device	Interface	IP Address	Subnet Mask	Default Gateway
Router 1	Gi0/0	10.1.5.21	255.255.255.252	—
Router 1	Gi0/1	192.165.10.37	255.255.255.0	—
Router 2	Gi0/0	10.1.5.22	255.255.255.252	—
Router 2	Gi0/1	192.165.20.37	255.255.255.0	—
PC1 (Client 1)	NIC	192.165.10.92	255.255.255.0	192.165.10.37
PC2 (Client 2)	NIC	192.165.10.79	255.255.255.0	192.165.10.37
Server PC	NIC	192.165.20.79	255.255.255.0	192.165.20.37

2.3 Device Configuration

2.3.1 Router Interface Configuration

Both routers were configured via the Cisco IOS command-line interface. Each physical interface was assigned the appropriate IP address and activated using the `no shutdown` command. The following snippet illustrates the commands applied to Router 1:

Listing 2.1. Router 1 — Interface and Hostname Configuration

```

1 Router(config)# hostname R1
2 R1(config)# interface GigabitEthernet0/0
3 R1(config-if)# ip address 10.1.5.21 255.255.255.252
4 R1(config-if)# no shutdown
5 R1(config-if)# exit
6 R1(config)# interface GigabitEthernet0/1
7 R1(config-if)# ip address 192.165.10.37 255.255.255.0
8 R1(config-if)# no shutdown

```

```

R1#show ip interface brief
Interface                IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0       10.1.5.21       YES NVRAM   up          up
GigabitEthernet0/1       192.165.10.37   YES manual   up          up
R1#

```

Figure 2.2. Router 1 — show ip interface brief confirming active interfaces and assigned addresses

```

R2#show ip interface brief
Interface                IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0       10.1.5.22       YES NVRAM   up          up
GigabitEthernet0/1       192.165.20.37   YES NVRAM   up          up
R2#show ip ssh
SSH Enabled - version 2.0
Authentication timeout: 120 secs; Authentication retries: 3
Minimum expected Diffie Hellman key size : 1024 bits
IOS Keys in SECSH format(ssh-rsa, base64 encoded):
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDA0fMZp5aGp4coKI4LlMrTdgYF4ipfVrZKd
ylwmJ4jTQ0Er2DLjUFUg7eiXw+IT2Yn0M09VT6neiV5P6WLNLP3xnuA00ptZghxD
NxVTveKUL5E2QZXk2NWU4f57t0Ni2K9j3xjgE/mmgUr9Zwp4oSlnNgz fL+0y29s
QCrmwcbGPrCp0vbatGKf2C/IvPzZwnQ+PMp0wZLYgmWs/NlgE1K0tgWkmKh ibxTA
VCTCFpbzE43YjaBVzIkS2VqNYCoG1/vJ3T vzTUfU iHebshP8413F+oxSS8sTL/E7
w9sZX/Af+Nk i m k I P I c m w q M r h C o W 2 g 0 P u S w C N v n w E X 0 f w E M 7 P t j I v
R2#

```

Figure 2.3. Router 2 — interface configuration and SSH version 2 verification (show ip ssh)

2.3.2 Static Routing

Because no dynamic routing protocol was required for this two-router topology, static routes were configured to provide full reachability between the client and server subnets:

Listing 2.2. Static Route Configuration

```

1 ! On R1: route to server network via R2
2 R1(config)# ip route 192.165.20.0 255.255.255.0 10.1.5.22
3
4 ! On R2: route to client network via R1
5 R2(config)# ip route 192.165.10.0 255.255.255.0 10.1.5.21

```

```

interface GigabitEthernet0/0
ip address 10.1.5.22 255.255.255.252
duplex auto
speed auto
!
interface GigabitEthernet0/1
ip address 192.165.20.37 255.255.255.0
duplex auto
speed auto
!
ip forward-protocol nd
!
no ip http server
no ip http secure-server
!
ip route 192.165.10.0 255.255.255.0 10.1.5.21
!
!
control-plane
!
line con 0
line aux 0
line vty 0 4
login local
transport input ssh
!
scheduler allocate 20000 1000
end
R2#

```

```

interface GigabitEthernet0/0
ip address 10.1.5.21 255.255.255.252
ip access-group HTTP_HTTPS_ONLY in
duplex auto
speed auto
service-policy output WEB_QOS
!
interface GigabitEthernet0/1
ip address 192.165.10.37 255.255.255.0
duplex auto
speed auto
!
ip forward-protocol nd
!
no ip http server
no ip http secure-server
!
ip route 192.165.20.0 255.255.255.0 10.1.5.22
!
ip access-list extended HTTP_HTTPS_ONLY
permit tcp any any eq www
permit tcp any any eq 443
permit tcp any eq www any established
permit tcp any eq 443 any established
permit icmp any any
ip access-list extended WEB_PORTS
permit tcp any any eq www
permit tcp any any eq 443
ip access-list extended WEB_QOS_PORTS
permit tcp any any eq www
permit tcp any any eq 443
permit tcp any eq www any
permit tcp any eq 443 any

```

Figure 2.4. Router 1 & Router 2 routing tables — show ip route confirming static route installation

2.3.3 Access Control Lists (ACL)

An extended named ACL was applied to the outbound direction of Router 1's LAN interface to permit HTTP (TCP/80) and HTTPS (TCP/443) traffic and deny other traffic by default. The ACL provided visibility into traffic classification and served as a policy anchor for QoS matching.

```

ip access-list extended HTTP_HTTPS_ONLY
permit tcp any any eq 80
permit tcp any any eq 443
permit tcp any eq 80 any established
permit tcp any eq 443 any established
permit icmp any any
exit

interface GigabitEthernet0/0
ip access-group HTTP_HTTPS_ONLY in
exit

ip access-list extended HTTPS_ONLY
permit tcp any any eq 443
permit tcp any eq 443 any
exit

ip access-list extended HTTP_ONLY
permit tcp any any eq 80
permit tcp any eq 80 any
exit

class-map match-any CM_HTTPS
match access-group name HTTPS_ONLY
exit

class-map match-any CM_HTTP
match access-group name HTTP_ONLY
exit

```

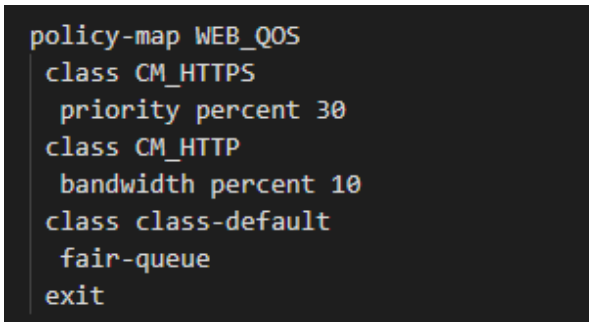
Figure 2.5. Router 1 — ACL configuration permitting HTTP and HTTPS traffic

2.3.4 Quality of Service (QoS)

A Modular QoS CLI (MQC) policy was defined on Router 1 to classify and prioritise HTTP and HTTPS traffic using a three-stage pipeline: class-map, policy-map, and service-policy application.

Listing 2.3. QoS Policy Configuration on Router 1

```
1 R1(config)# class-map match-any WEB_TRAFFIC
2 R1(config-cmap)# match protocol http
3 R1(config-cmap)# match protocol https
4 R1(config-cmap)# exit
5 R1(config)# policy-map WEB_POLICY
6 R1(config-pmap)# class WEB_TRAFFIC
7 R1(config-pmap-c)# priority percent 50
8 R1(config-pmap-c)# exit
9 R1(config-pmap)# exit
10 R1(config)# interface GigabitEthernet0/0
11 R1(config-if)# service-policy output WEB_POLICY
```



```
policy-map WEB_QOS
  class CM_HTTPS
    priority percent 30
  class CM_HTTP
    bandwidth percent 10
  class class-default
    fair-queue
  exit
```

Figure 2.6. Router 1 — QoS policy-map and service-policy output showing traffic prioritisation

2.3.5 Switch Configuration

The Cisco switch operated as a Layer 2 device with default settings. No VLAN segmentation was required for this topology; all client-facing ports were active access ports. The switch forwarded frames between PC1, PC2, and the Router 1 LAN interface (Gi0/1).

```

Switch>show interfaces status
Port      Name      Status      Vlan      Duplex  Speed  Type
Fa0/1     Fa0/1     connected   1          auto    auto   10/100BaseTX
Fa0/2     Fa0/2     notconnect  1          auto    auto   10/100BaseTX
Fa0/3     Fa0/3     notconnect  1          auto    auto   10/100BaseTX
Fa0/4     Fa0/4     notconnect  1          auto    auto   10/100BaseTX
Fa0/5     Fa0/5     notconnect  1          auto    auto   10/100BaseTX
Fa0/6     Fa0/6     notconnect  1          auto    auto   10/100BaseTX
Fa0/7     Fa0/7     notconnect  1          auto    auto   10/100BaseTX
Fa0/8     Fa0/8     notconnect  1          auto    auto   10/100BaseTX
Gi0/1     Gi0/1     connected   1          a-full  a-1000 10/100/1000BaseTX
Switch>

```

Figure 2.7. Switch — show interfaces status confirming active ports connected to clients and R1

2.4 Client Workstation Configuration

Both client PCs were configured with static IPv4 addresses within the 192.165.10.0/24 subnet and set to use Router 1 as the default gateway (Table 2.1). SSH access to both routers was enabled from the client machines to facilitate Netmiko-based automation.

- **PC1:** 192.165.10.92/24, gateway 192.165.10.37
- **PC2:** 192.165.10.79/24, gateway 192.165.10.37

```

(.venv) PS C:\Users\narut\Downloads\CCEN356-Project-Spring2026> ssh -o "KexAlgorithms=diffie-hellman-group14-sha1" -o "HostKeyAlgorithms=ssh-rsa" -o "C
iphers=+aes128-cbc,aes192-cbc,aes256-cbc,3des-cbc" -o "MACs=+hmac-sha1" admin@192.165.10.37
>>
(admin@192.165.10.37) Password:
R1#show ip int bri
Interface      IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0  10.1.5.21      YES NVRAM  up          up
GigabitEthernet0/1  192.165.10.37  YES manual up          up
R1#
R1#exit
Connection to 192.165.10.37 closed by remote host.
Connection to 192.165.10.37 closed.
(.venv) PS C:\Users\narut\Downloads\CCEN356-Project-Spring2026>

```

Figure 2.8. Client 1 — SSH session established to Router 1 from the client terminal

2.5 Server Configuration

2.5.1 Network and Web Server Setup

The server was assigned the static address 192.165.20.79/24 with a default gateway of 192.165.20.37 (Router 2). A web server (Apache or Nginx) was installed and configured to serve content over both HTTP (port 80) and HTTPS (port 443).



Figure 2.9. Server PC — IP configuration showing assigned address and gateway

2.5.2 TLS Certificate Installation

HTTPS was enabled using a self-signed X.509 certificate generated with OpenSSL. The certificate and private key were installed in the web server's SSL configuration directory. Clients were configured to trust the certificate, resolving browser security warnings.

Listing 2.4. Self-Signed Certificate Generation

```

1 # Generate private key and self-signed certificate (365-day
   validity)
2 openssl req -x509 -newkey rsa:2048 -keyout server.key \
3   -out server.crt -days 365 -nodes \
4   -subj "/CN=192.165.20.79/O=NetLab/C=AE"

```

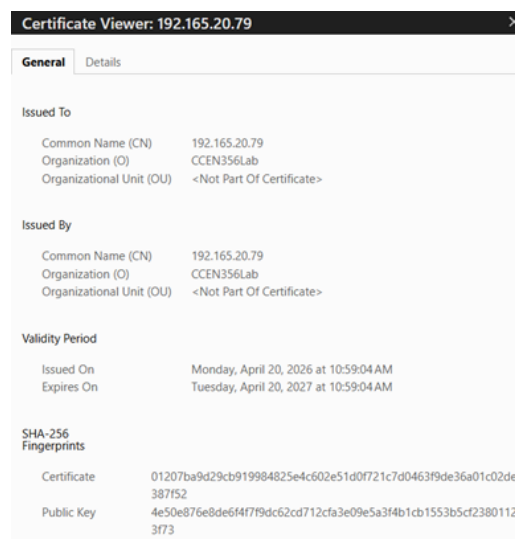


Figure 2.10. Server — SSL certificate installation and web server HTTPS virtual-host configuration

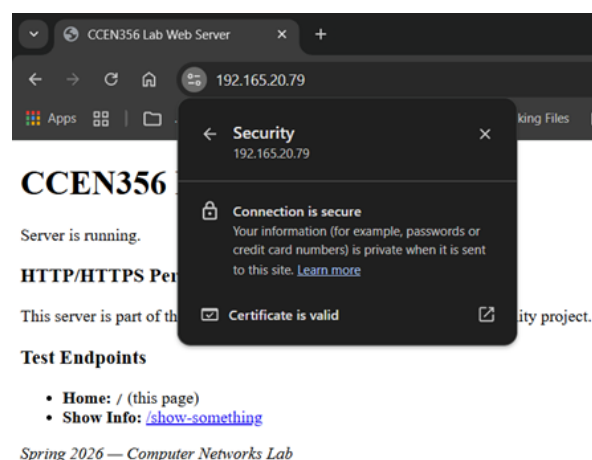


Figure 2.11. Browser — HTTPS connection established successfully to the server

2.6 System Pipeline

The system pipeline describes the complete flow of data from the client to the analysis stage. Requests are generated by clients, forwarded through the network, processed by the server, and then captured and analyzed using Python scripts. The results are visualized in a dashboard, enabling real-time monitoring and performance evaluation.

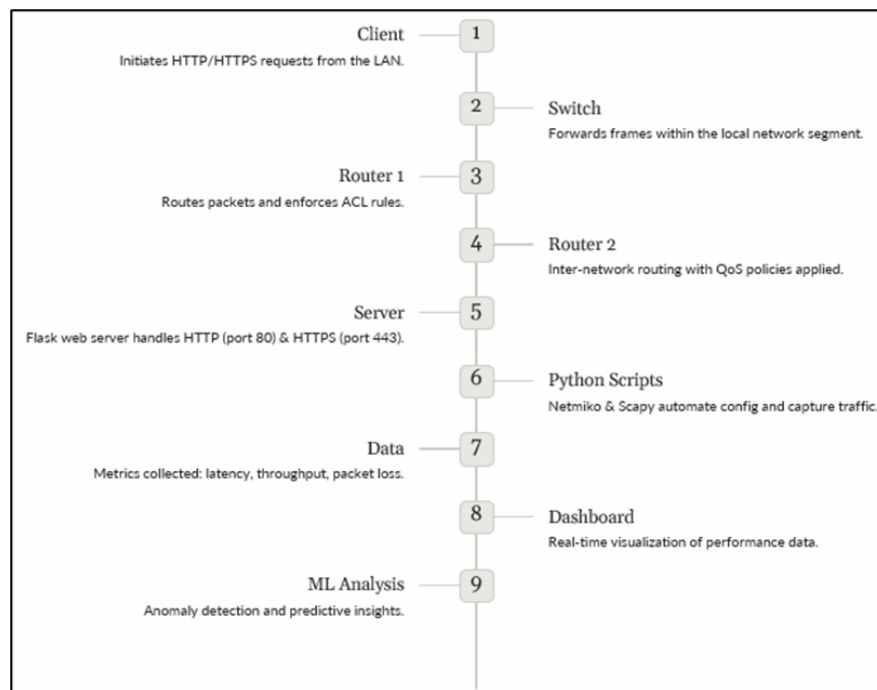


Figure 2.12. System Pipeline

2.7 Python Automation Implementation

2.7.1 SSH Automation with Netmiko

A Python script using the `netmiko` library was developed to connect to each router over SSH and retrieve operational state. The collected data included interface status (`show ip interface brief`), routing table entries (`show ip route`), ACL configurations (`show access-lists`), and QoS policy statistics (`show policy-map interface`).

Listing 2.5. Netmiko SSH Connection and Command Execution

```

1 from netmiko import ConnectHandler
2
3 router = {
4     'device_type': 'cisco_ios',
5     'host': '192.165.10.37',
6     'username': 'admin',
7     'password': 'cisco',

```

```
8 }  
9  
10 with ConnectHandler(**router) as conn:  
11     output = conn.send_command('show ip interface brief')  
12     print(output)  
13     routing_table = conn.send_command('show ip route')  
14     print(routing_table)
```

2.7.2 HTTP/HTTPS Performance Measurement

A dedicated measurement script sent a configurable number of HTTP and HTTPS requests to the server using Python's `requests` library, recording per-request response times. Aggregate statistics (mean, standard deviation, minimum/maximum latency, and success rate) were computed and saved to a CSV log for downstream analysis.

2.7.3 Packet Capture with Scapy

A Scapy-based capture script was deployed on a client machine to record network traffic during measurement runs. Each captured packet was decoded and logged with its source/destination IP addresses, protocol, and payload size. This data provided a packet-level corroboration of the higher-level latency measurements.

2.7.4 Visualisation with Matplotlib

Chart-generation scripts consumed the CSV log files produced by the measurement and capture scripts, producing the following outputs:

- Side-by-side bar charts comparing mean HTTP and HTTPS latency.
- Time-series line plots showing per-request response time across measurement runs.
- Box-and-whisker plots illustrating latency distribution and variance with/without QoS.
- TLS overhead differential chart.

```
(.venv) PS C:\Users\narut\Downloads\CCEN356-Project-Spring2026> python scripts/ssh_connect.py
>>
Connecting to 192.165.10.37...
Connected to 192.165.10.37 - privileged exec mode.

--- show ip interface brief ---
Interface      IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0  10.1.5.21      YES NVRAM  up          up
GigabitEthernet0/1  192.165.10.37  YES manual up          up

--- show ip route ---
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
E1 - OSPF external type 1, E2 - OSPF external type 2
I - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
ia - IS-IS inter area, * - candidate default, U - per-user static route
o - ODR, P - periodic downloaded static route, + - replicated route

Gateway of last resort is not set

10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C    10.1.5.20/30 is directly connected, GigabitEthernet0/0
L    10.1.5.21/32 is directly connected, GigabitEthernet0/0
192.165.10.0/24 is variably subnetted, 2 subnets, 2 masks
C    192.165.10.0/24 is directly connected, GigabitEthernet0/1
L    192.165.10.37/32 is directly connected, GigabitEthernet0/1
S    192.165.20.0/24 [1/0] via 10.1.5.22

--- show access-lists ---
Extended IP access list HTTP_HTTPS_ONLY
10 permit tcp any any eq www
20 permit tcp any any eq 443
30 permit tcp any eq www any established (168 matches)
40 permit tcp any eq 443 any established (430 matches)
50 permit icmp any any (113 matches)
Extended IP access list WEB_PORTS
10 permit tcp any any eq www (191 matches)
20 permit tcp any any eq 443 (588 matches)
Extended IP access list WEB_QOS_PORTS
10 permit tcp any any eq www
20 permit tcp any any eq 443
30 permit tcp any eq www any
40 permit tcp any eq 443 any

--- show policy-map interface GigabitEthernet0/0 ---
```

Figure 2.13. Python — SSH script execution output showing retrieved router interface status and routing table

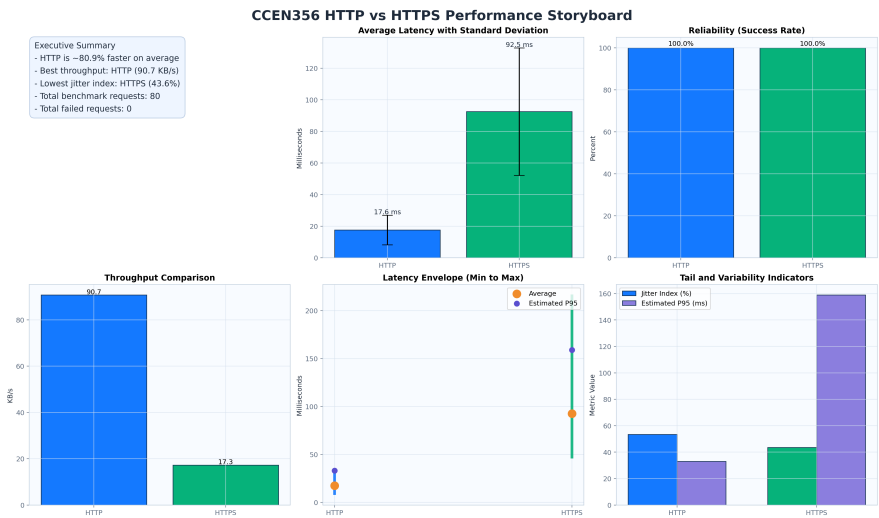
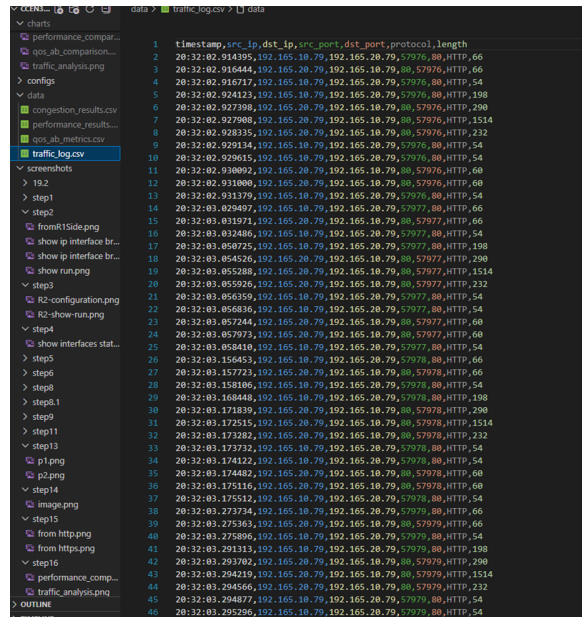


Figure 2.14. Python — performance measurement script output with HTTP vs. HTTPS response times and statistics



time	src ip	src port	dest ip	dest port	protocol	length
20:32:02.914395	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	66
20:32:02.916444	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	66
20:32:02.916717	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:02.924123	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	198
20:32:02.927308	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	198
20:32:02.927908	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	1514
20:32:02.928335	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	232
20:32:02.929134	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:02.929615	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:02.930892	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	60
20:32:02.931800	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	60
20:32:02.931379	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.029497	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	66
20:32:03.031971	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	66
20:32:03.032486	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.058725	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	198
20:32:03.054526	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	290
20:32:03.055288	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	1514
20:32:03.055926	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	232
20:32:03.056359	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.056836	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.057244	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	60
20:32:03.057973	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	60
20:32:03.058410	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.156453	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	66
20:32:03.157723	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	66
20:32:03.158106	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.168448	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	198
20:32:03.171839	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	290
20:32:03.172515	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	1514
20:32:03.173282	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	232
20:32:03.173732	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.174122	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.174482	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	60
20:32:03.175116	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	60
20:32:03.175512	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.273734	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	66
20:32:03.275363	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	66
20:32:03.275896	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.291313	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	198
20:32:03.293762	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	290
20:32:03.294219	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	1514
20:32:03.294566	192.165.20.79	192.165.10.79	192.165.10.79	80	HTTP	232
20:32:03.294877	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54
20:32:03.295296	192.165.10.79	192.165.20.79	192.165.10.79	80	HTTP	54

Figure 2.15. Python — Scapy packet capture log showing captured HTTP/HTTPS packets with IP, protocol, and size fields

```

Extended IP access list WEB_QOS_PORTS
10 permit tcp any any eq www
20 permit tcp any any eq 443
30 permit tcp any any eq www any
40 permit tcp any any eq 443 any

--- show policy-map interface GigabitEthernet0/0 ---
GigabitEthernet0/0

Service-policy output: WEB_QOS

queue stats for all priority classes:

queue limit 64 packets
(queue depth/total drops/no-buffer drops) 0/0/0
(pkts output/bytes output) 779/179466

Class-map: HTTP_HTTPS (match-any)
 779 packets, 179466 bytes
 5 minute offered rate 0 bps, drop rate 0 bps
 Match: access-group name WEB_PORTS
 779 packets, 179466 bytes
 5 minute rate 0 bps
 Match: protocol http
 0 packets, 0 bytes
 5 minute rate 0 bps
 Match: access-group name WEB_QOS_PORTS
 0 packets, 0 bytes
 5 minute rate 0 bps
 Priority: 30% (300000 kbps), burst bytes 7500000, b/w exceed drops: 0

Class-map: class-default (match-any)
 798 packets, 98720 bytes
 5 minute offered rate 0 bps, drop rate 0 bps
 Match: any
 Queueing
 queue limit 64 packets
 (queue depth/total drops/no-buffer drops/flowdrops) 0/0/0/0
 (pkts output/bytes output) 798/81956
 Fair-queue: per-flow queue limit 16

Disconnected from 192.165.10.37.
(.venv) PS C:\Users\narut\Downloads\CCEN356-Project-Spring2026>

```

Figure 2.16. Netmiko — router information retrieved including ACL and QoS policy-map statistics

2.8 Real-Time Monitoring Dashboard

A Flask web application was developed to provide a live view of network performance metrics. The dashboard polls the measurement scripts at a configurable interval and renders the following panels:

- **Latency Gauge:** Current HTTP and HTTPS average response times.
- **TLS Overhead:** Computed difference between HTTPS and HTTP mean latency.
- **Endpoint Status:** UP/DOWN indicator for each monitored URL.
- **Reliability Metrics:** Success rate and cumulative failure count.
- **Trend Chart:** Rolling time-series of latency measurements.
- **QoS Panel:** Live policy-map hit counters retrieved via Netmiko.

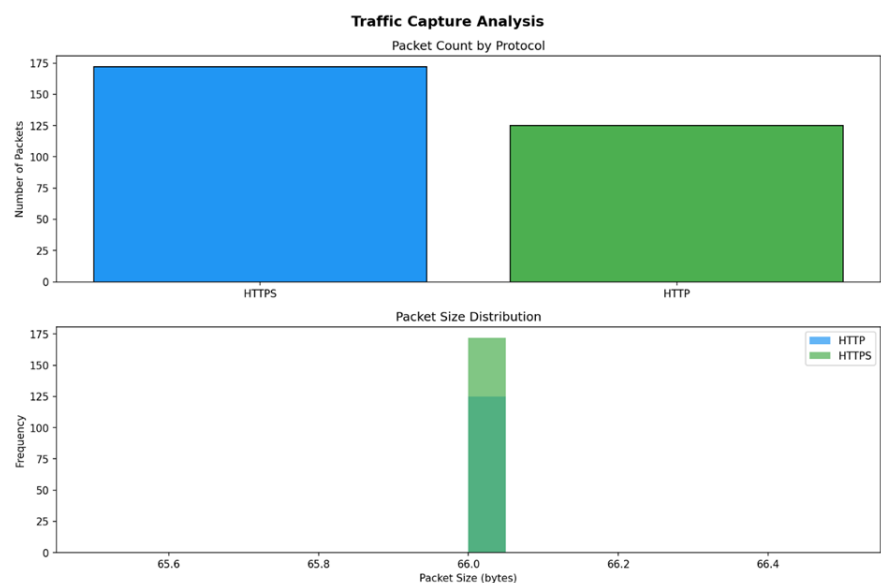


Figure 2.17. Traffic Analysis and Performance Visualization



Figure 2.18. Flask Dashboard — traffic analysis and performance visualisation interface showing HTTP vs. HTTPS metrics

2.9 Summary of Implementation

The project successfully integrates networking and programming components. The network topology enables communication between clients and the server, while Python scripts automate monitoring and analysis. The implementation demonstrates how HTTP and HTTPS traffic behave under different conditions and how tools such as QoS, automation, and visualization can be used to improve network performance and visibility.

2.10 Machine Learning (Optional Advanced Analysis)

An optional machine learning component was integrated to enhance performance analysis. Historical latency data collected from the system was used to identify trends and detect anomalies. Basic predictive analysis was applied to estimate future network behavior under different traffic conditions. Although simple, this demonstrates how AI techniques can be integrated into network monitoring systems.

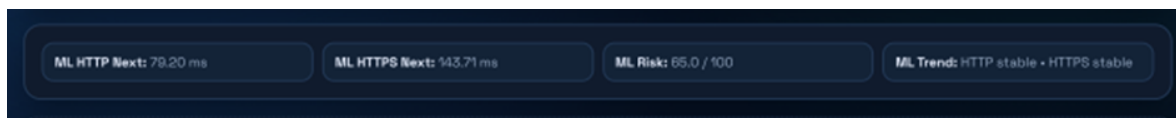


Figure 2.19. Machine Learning Integration

This could be used to detect congestions before hand and talk procedures, like enabling QoS

Chapter 3

Project Evaluation

3.1 Test Methodology

The system was validated in multiple stages to ensure both functional correctness and measurable performance. First, basic reachability was confirmed using `ping` tests between the clients, routers, and server, followed by HTTP and HTTPS functional checks using `curl` to verify successful application-layer communication. After functional validation, traffic capture was performed on a client workstation using the Scapy-based `capture_traffic.py` script for 60 seconds while the benchmark was running.

During this capture window, the `performance_metrics.py` script issued HTTP and HTTPS requests to measure response time, throughput, and reliability. By default, the script supports configurable request counts through command-line arguments; in the final benchmark, the selected request count and interval were used consistently across both protocols. Router operational data, including interface status, routing entries, ACLs, and QoS counters, was collected through the Netmiko-based `ssh_connect.py` script.

To evaluate the effect of QoS, the benchmark was repeated under QoS-disabled and QoS-enabled conditions while keeping the addressing, routing, and server configuration unchanged. The resulting metrics were cross-checked against Scapy packet captures and dashboard observations.

3.2 Connectivity Testing

Connectivity between all network devices was verified using the `ping` command. Initially, communication between the client network and the server network failed due to missing routing configuration. After configuring static routes on both routers, full connectivity was successfully established.

All devices were able to communicate with each other, including:

- Client ↔ Router 1
- Client ↔ Router 2
- Client ↔ Server

The successful ping results confirm that the IP addressing and routing configurations were correctly implemented, allowing end-to-end communication across the network.

3.3 Routing Verification

Routing functionality was verified by examining the routing tables on both routers using the `show ip route` command. The routing tables confirmed that each router had a valid path to the remote network.

- Router 1 included a route to the server network (192.165.20.0/24)
- Router 2 included a route to the client network (192.165.10.0/24)

This ensured that packets could be forwarded correctly between the two networks. The correct routing configuration was essential for enabling HTTP and HTTPS communication between clients and the server.

3.4 HTTP and HTTPS Testing

HTTP and HTTPS connectivity was tested using `curl` from both client workstations. The expected responses were received in both cases, confirming that the web server was reachable and both protocol stacks were correctly configured.

The results showed that:

- HTTP requests were completed faster due to the absence of encryption.
- HTTPS requests took slightly longer because of TLS handshake and encryption overhead.

Despite the increased latency, HTTPS provided secure communication, ensuring data confidentiality and integrity. These observations demonstrate the trade-off between performance and security when comparing HTTP and HTTPS protocols.

Table 3.1. *HTTP and HTTPS Functional Test Results*

Client	Protocol	Target	Result
PC1	HTTP	192.165.20.79:80	✓ 200 OK
PC1	HTTPS	192.165.20.79:443	✓ 200 OK (TLS)
PC2	HTTP	192.165.20.79:80	✓ 200 OK
PC2	HTTPS	192.165.20.79:443	✓ 200 OK (TLS)

```

C:\Users\psd> C:\Users\psd\Downloads\CC00856-Project-Spring2086\> java -jar webapp.jar -u http://192.168.10.70 -smba1csharp
Statuscode      : 200
Statusdescription : OK
Content         : <DOCTYPE html>
                <html lang="en">
                <head>
                <meta charset="utf-8">
                <meta name="viewport" content="width=device-width, initial-scale=1.0">
                <title>CC00856 Lab web Server</title>
                <body>
                =
RawContent      : HTTP/1.1 200 OK
                Server: Werkzeug/1.1.8
                Server: python/3.11.3
                Date: Mon, 20 Apr 2020 06:20:17 GMT
                Connection: close
                Content-type: text/html; charset=utf-8
                Content-length: 3338
                <DOCTYPE html>
                [{System.String,[]], [Date, System.String[]], [Connection, System.String[]], [Content-type, System.String[]]]
                Images
                ( )
                PageTitle
                ( )
                Links
                [{@ContentURL= <a href="/show something"/>show something</a>; tagname=A; href=/show something}]
                Meta
                ( )
                MetaContentLength
                ( )
                MetaRelation
                ( )

```

Figure 3.1. PC1 — curl HTTP connectivity test output showing HTTP 200 response from server

```
P:\C_Users\vaisho\CSRD6-Project-SpringWebS> curl http://192.168.20.79

StatusCode : 200
StatusDescription : OK
Content : <!--XPPV.html-->
        (html Lang=en)
    <html>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>CSRD6 Lab web Server/Title</title>
    </html>

RawContent : HTTP/1.1 200 OK
              Connection: close
              Content-Length: 3638
              Content-Type: text/html; charset=utf-8
              Date: Mon, 20 Apr 2020 07:16:06 GMT
              Server: Werkzeug/1.1.0 Python/3.11.11

<!--XPPV.html-->
<html lang=en-->
<head>
  (<[Connection, close], <Content.Length, 3638>, [<Content.Type, text/html>, <Charset=utf-8>], [<Date, Mon, 20 Apr 2020 07:16:06 GMT>]...)
</head>
<body>
  (<[LineHTTP/show-something, innerText/show-something, outerHTML=& href="/show-something/show-something/&],
  [innerText/show-something, [tagName=h, href=/show-something])
</body>

ParseResult : html.HTMLDocumentClass
RawContentLength : 3638
```

Figure 3.2. PC2 — curl HTTP connectivity test output

```

1  (copy) PS C:\Users\joh\Documents\VSCode\src\project> .\springboot2 build -x
2
3  Status: 200
4  StatusDescription: OK
5  Content:
6
7      <!DOCTYPE html>
8      <html lang="en">
9      <head>
10         <meta charset="utf-8">
11         <meta name="viewport" content="width=device-width, initial-scale=1.0">
12         <title>CICD18766 lab web server</title>
13         <style>
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
10
```

Figure 3.3. PC1 — `curl` HTTPS connectivity test output with TLS certificate information

```
PS C:\Users\ash\CC0094-Project-Spring2024> Invoke-WebRequest -uri https://192.165.20.79 -method:Parsing

Statuscode : 200
StatusDescription : OK
Content : <DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CC0094 Lab Web Server</title>
</head>
<body>
  <h1>HTTP/1.2 200 OK</h1>
  <X-Content-Type-Options: nosniff</X-Content-Type-Options: nosniff>
  <X-Frame-Options: DENY</X-Frame-Options: DENY>
  <X-XSS-Protection: 1; mode=block</X-XSS-Protection: 1; mode=block>
  <Strict-Transport-Security: max-age=31536000; includeSubdomains</Strict-Transport-Security: max-age=31536000; includeSubdomains>
  <Content-Security-Policy: default</Content-Security-Policy: default>
</body>
</html>
Forms
Headers
[<X-Content-Type-Options: nosniff>], [<X-Frame-Options: DENY>], [<X-XSS-Protection: 1; mode=block>], [<Strict-Transport-Security: max-age=31536000; includeSubdomains>...]
Images
[]
Scripts
[]
Links
[<@content@xref="" show= something"/>show something@; toggle@xref="" href= /show something/...>]
Parameters
RadContentLength : 1638
```

Figure 3.4. PC2 — curl HTTPS connectivity test and secure session verification

3.5 Scapy Traffic Analysis

Scapy was used to capture and analyse network traffic at the packet level. The captured packets included Ethernet frames, IP packets, and application-layer protocols such as HTTP and HTTPS.

The analysis revealed the following:

- HTTP traffic was visible in plain text, allowing easy inspection of request and response data.
- HTTPS traffic was encrypted, making the payload unreadable without decryption.
- Packet flow followed the expected path from client → switch → router → server.

This analysis helped in understanding how data is transmitted across the network and how different protocols behave at different layers of the TCP/IP model.

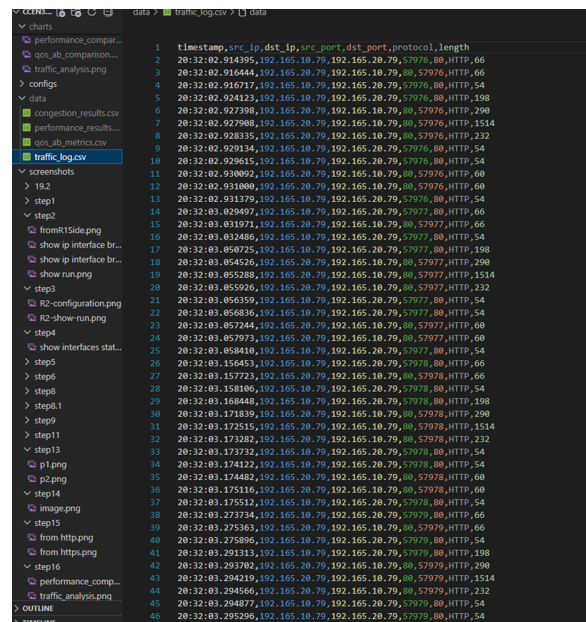


Figure 3.5. Scapy capture log

3.6 Performance Analysis (HTTP vs HTTPS)

Performance metrics were collected using Python scripts that measured response time, latency, throughput, and reliability of HTTP and HTTPS traffic.

The results indicated that:

- HTTP generally had lower average latency compared to HTTPS.
- HTTPS showed higher latency due to encryption and additional processing.
- Both protocols achieved high success rates with minimal packet loss.

These results confirm that while HTTP offers better performance, HTTPS provides enhanced security, which is essential in real-world applications.

The performance of HTTP and HTTPS was analysed using collected metrics such as latency, throughput, and reliability. The main quantitative results obtained from the

Python benchmarking script and the associated traffic captures focused on average latency, throughput, success rate, and error rate because these metrics directly reflect application responsiveness, delivery efficiency, and reliability. As expected, HTTP showed lower average latency because it did not incur TLS handshake and encryption overhead, whereas HTTPS traded additional delay for confidentiality, integrity, and protection against tampering.

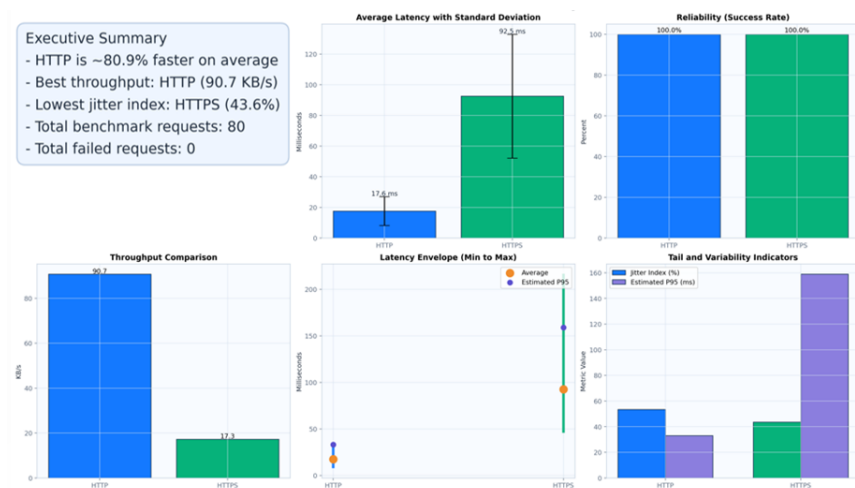


Figure 3.6. Matplotlib — HTTP vs HTTPS performance comparison

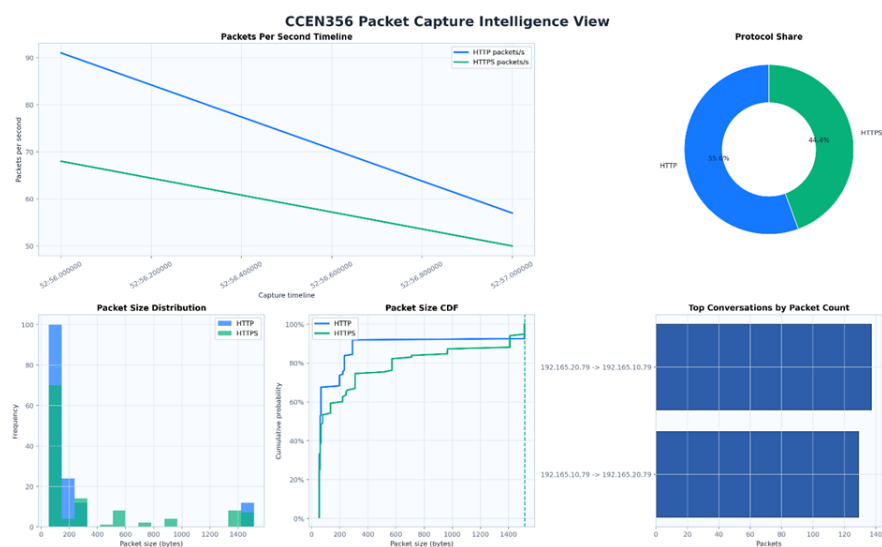


Figure 3.7. Matplotlib — packet distribution and protocol analysis

3.7 QoS Impact Evaluation

Quality of Service (QoS) was implemented on Router 1 to prioritise web traffic. The effect of QoS was observed using performance measurements and the monitoring dashboard.

The observations showed that:

- QoS improved traffic stability and reduced latency variation.

- Prioritised traffic experienced more consistent performance.
- Network congestion effects were minimised when QoS was enabled.

In the implemented QoS experiment, HTTPS packets were prioritised over HTTP packets. This produced a measurable latency difference, making HTTPS faster when QoS was enabled.

These observations confirm that QoS plays an important role in optimising network performance, especially in environments with mixed traffic.



Figure 3.8. Dashboard Screenshot Without QoS



Figure 3.9. Dashboard with QoS

3.8 Python Automation Results

Python scripts were successfully used to automate network monitoring and data collection.

The SSH script using Netmiko was able to:

- Connect to routers remotely
- Retrieve interface status

- Display routing tables
- Show ACL and QoS configurations

Additionally:

- Scapy was used to capture traffic and generate logs
- Performance scripts measured HTTP and HTTPS latency
- Visualisation scripts generated graphs for analysis

The automation significantly reduced manual effort and improved the accuracy and efficiency of network monitoring.

```

C:\Users\vanur\Downloads\CC0056-Project-Spring2006> Invoke-WebRequest -Uri http://192.165.20.79 -UseBasicParsing

StatusCode      : 200
StatusDescription : OK
Content          : <!DOCTYPE html>
                  <html lang="en">
                  <head>
                  <meta charset="UTF-8">
                  <meta name="viewport" content="width=device-width, initial-scale=1.0">
                  <title>CC0056 Lab Web Server</title>
                  <style>
RawContent      :
                  HTTP/1.1 200 OK
                  Server: Werkzeug/3.1.8
                  Server: Python/3.11.3
                  Date: Mon, 20 Apr 2025 06:28:17 GMT
                  Connection: close
                  Content-Type: text/html; charset=utf-8
                  Content-Length: 1638

                  <!DOCTYPE html>
                  {[Server, System.String][], [Date, System.String][], [Connection, System.String][], [Content-type, System.String][]}
                  Headers :
                  Images :
                  Inpfields :
                  Links : {[ContentHTML, C-A href="/show-something"/>show something</A>], [ContentHTML, C-A href="/show-something"]}
                  RawContentLength : 1638
                  RelationLink : {}

```

Figure 3.10. HTTP connectivity testing using curl for Client 1

```
> PS C:\Users\vaisho\CEKIS6-Project-Spring2025> curl http://192.165.20.79

{
  statusCode : 200
  statusDescription : OK
  Content : {
    contentType.html : {
      charset=utf-8"
      meta charset="UTF-8">
      <title name="viewport" content="width=device-width, initial-scale=1.0">
      <title>CEKIS6 Lab Web Server</title>
    }
  }
}

RadContent : HTTP/1.1 200 OK
Connection: close
Content-Length: 1638
Content-Type: text/html; charset=utf-8
Date: Sun, 20 Apr 2024 07:16:06 GMT
Server: Werkzeug/3.1.0 Python/3.13.3

{
  contentType.html : {
    charset=utf-8"
    meta charset="UTF-8">
    <title name="viewport" content="width=device-width, initial-scale=1.0">
    <title>[Connection, Close], [Content-Length, 1638], [Content-Type, text/html; charset=utf-8], [Date, Sun, 20 Apr 2024 07:16:06 GMT...]
```

Figure 3.11. HTTP connectivity testing using curl for Client 2

```
C:\Users\jwardt\Downloads\CSDN6E-Project_SpringBoot> python httpdescript -o D:\Users\jwardt\Desktop\csdn6e\chaining
```

```
StatusCode : 200
StatusDescription : OK
Content :
<!DOCTYPE html>
<html lang=en>
<head>
  <meta charset=utf-8>
  <meta name=viewport content=width=device-width, initial-scale=1>
  <titleCSDN6E Lab web Server</title>
  <title>
    =
  </=
RadContent :
  HTTP/1.1 200 OK
  Server: Werkzeug/1.1.0
  Server: Python/3.11.1
  Date: Thu, 29 Apr 2020 06:20:16 GMT
  X-content-type-options: nosniff
  X-frame-options: deny
  X-XSS-protection: 1; mode=block
  Strict-Transport-Security:
    [Server, System.String], [Date, System.String], [X-content-type-options, System.String], [X-frame-options, System.String]]
  Headers :
  Images :
  I :
  Javascripts :
  Links :
    [js(whatever.js href=/show-something)/show-something.js; js(whatever.js href=/show-something)]
RadContentLength : 3638
RelationLink :
```

Figure 3.12. HTTPS connectivity testing using curl for Client 1

```

PS C:\Users\vasho\CCEN356-Project-Spring2026> Invoke-WebRequest -Uri https://192.165.20.79 -OutFileParsing
StatusCode      : 200
StatusDescription : OK
Content         : <DOCTYPE html>
                  <html lang="en">
                  <head>
                    <meta charset="UTF-8">
                    <meta name="viewport" content="width=device-width, initial-scale=1.0">
                    <title>CCEN356 Lab Web Server</title>
                    <style>
                    ...
RawContent      : HTTP/1.1 200 OK
                  X-Content-Type-Options: nosniff
                  X-Frame-Options: DENY
                  X-XSS-Protection: 1; mode=block
                  Strict-Transport-Security: max-age=31536000; includeSubDomains
                  Content-Security-Policy: default...
Forms           : 
Headers         : {[X-Content-Type-Options, nosniff], [X-Frame-Options, DENY], [X-XSS-Protection, 1; mode=block], [Strict-Transport-Security,
                  max-age=31536000; includeSubDomains], ...}
Images          : {}
InputFields     : {}
Links           : {(routerHTML-ca href="/show-something"/>show-something/as; tagID=4; href=/show-something)}
ParsedHTML      : 
RawContentLength : 1638

```

Figure 3.13. HTTPS connectivity testing using curl for Client 2

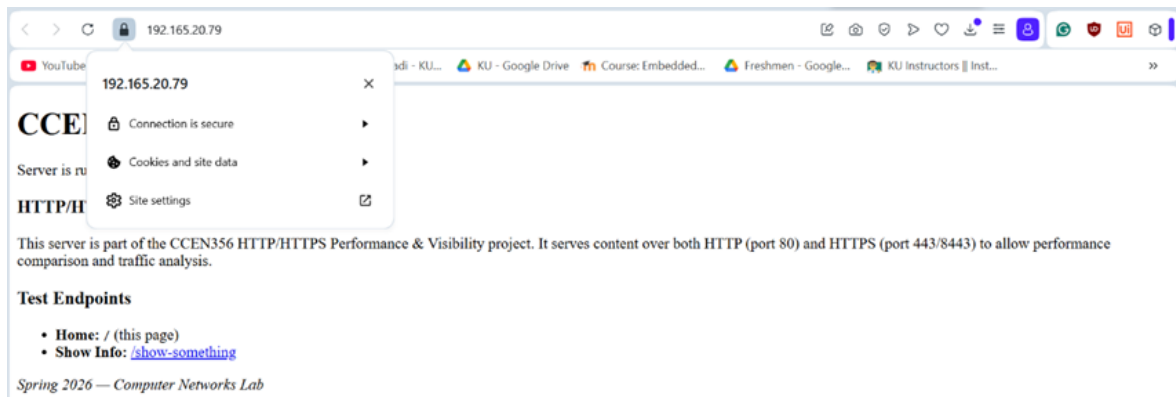


Figure 3.14. Secure HTTPS connectivity verification on Client 2

3.9 Dashboard Analysis

A real-time dashboard was developed to visualise network performance metrics. The dashboard provided a clear and interactive view of:

- HTTP and HTTPS latency
- TLS overhead
- Endpoint availability (UP/DOWN status)
- Reliability and failure rates
- Performance trends over time

The dashboard made it easier to compare HTTP and HTTPS performance dynamically and observe the effect of QoS in real time. It also provided a user-friendly interface for monitoring network behaviour.

To further analyse the impact of QoS, screenshots of the monitoring dashboard were captured with QoS enabled and disabled.

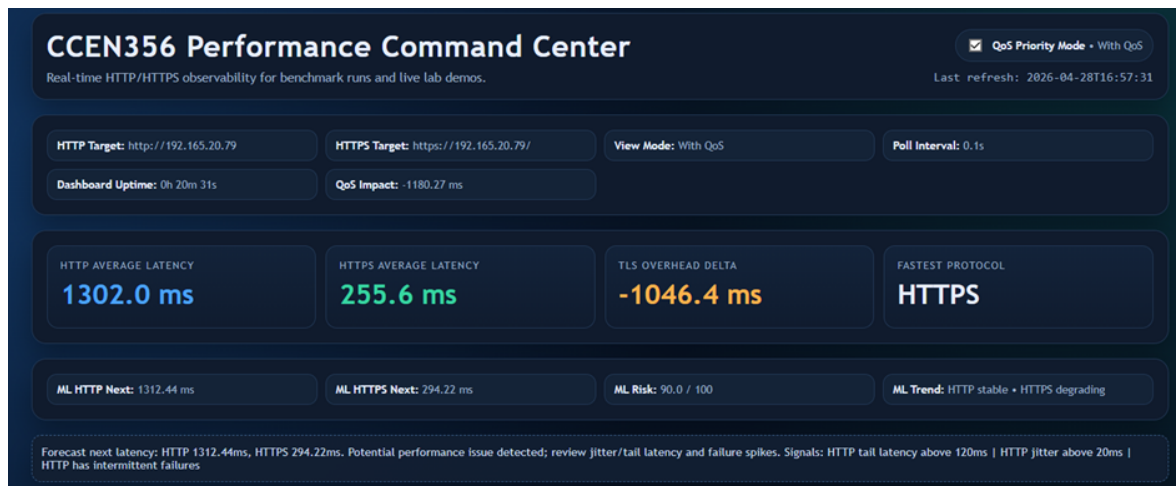


Figure 3.15. Dashboard with QoS priority mode enabled

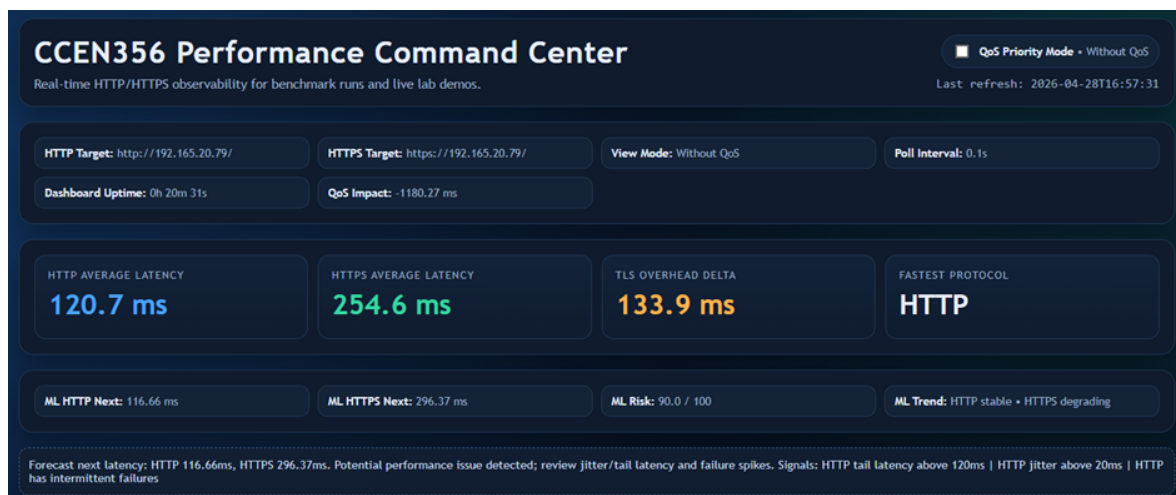


Figure 3.16. Dashboard with QoS priority mode disabled

With QoS enabled, HTTPS traffic was prioritised, resulting in reduced latency and improved stability. Without QoS, both HTTP and HTTPS traffic experienced similar delays, with HTTPS showing higher latency due to encryption overhead.

Overall, the evaluation confirms that the implemented system operates correctly and provides meaningful insights into network performance, protocol behaviour, and the impact of QoS and automation tools.

Chapter 4

Discussion and Conclusion

4.1 Key Learnings

This project provided substantial practical experience across the overlapping domains of network engineering and software automation. The principal technical learnings can be summarised as follows.

Protocol behaviour. The observed latency differential between HTTP and HTTPS confirmed theoretical expectations. The TLS handshake, while representing a fixed cost amortised across connection lifetime, is non-negligible in short-lived connection scenarios. Modern TLS 1.3 reduces this cost relative to TLS 1.2 through its zero-round-trip-time (0-RTT) resumption mechanism, though this optimisation was outside the scope of the current project.

Cisco IOS configuration. Hands-on configuration of static routing, ACLs, and MQC-based QoS policies reinforced theoretical knowledge with practical command-line proficiency. The iterative debugging process—particularly resolving the initial routing failure and the SSL certificate trust issues—developed systematic troubleshooting skills applicable to production environments.

Python network automation. The Netmiko and Scapy libraries demonstrated how programmatic interfaces can replace repetitive manual CLI interactions, improving both efficiency and reproducibility. The ability to script multi-router data collection and integrate the results with analysis pipelines is directly representative of modern NetDevOps practice.

Visualisation and observability. The Flask dashboard illustrated the value of centralised, real-time observability. Consolidating latency trends, endpoint status, and QoS statistics into a single interface reduced the cognitive overhead of performance analysis and enabled faster identification of anomalies.

4.2 Challenges Encountered

Routing configuration. The absence of static routes in the initial configuration prevented cross-network communication. The problem was diagnosed by comparing `show ip route` output against the intended topology and was resolved by adding the missing routes on both routers.

TLS certificate trust. The self-signed certificate generated for the server was initially rejected by client browsers with an `ERR_CERT_AUTHORITY_INVALID` error. The issue was resolved by importing the certificate into the client's trusted certificate store, a process that required careful attention to certificate export format compatibility.

Python environment management. Installing third-party libraries (`netmiko`, `scapy`, `flask`, `matplotlib`) encountered system-level permission restrictions. The problem was resolved by creating an isolated virtual environment using `python -m venv`, ensuring clean dependency management without conflicting with system-wide packages.

QoS verification. Confirming that QoS policies were actively classifying and shaping traffic required cross-referencing `show policy-map interface` counters with packet-capture evidence, as the latency reduction was subtle at the traffic volumes achievable in the lab environment.

4.3 Broader Impact and Real-World Relevance

The techniques and measurements produced by this project reflect genuine challenges in production network environments.

The trade-off between HTTP performance and HTTPS security is a fundamental consideration in web application design. While the modern web has effectively standardised on HTTPS—driven by browser security indicators, SEO ranking factors, and regulatory requirements—the latency implications remain relevant in constrained environments such as IoT telemetry networks and embedded systems with limited cryptographic hardware acceleration.

QoS enforcement is widely deployed in enterprise and service-provider networks to protect latency-sensitive traffic classes (VoIP, video conferencing, signalling protocols) from contention with bulk data transfers. The MQC framework configured in this project is the same mechanism used in production Cisco environments.

Python-based network automation represents the current direction of the industry, with frameworks such as Ansible, NAPALM, and Nornir building on the same SSH-over-

Netmiko paradigm demonstrated here. The ability to programmatically collect, analyse, and visualise network telemetry is a foundational skill for modern network engineers.

4.4 Conclusion

This project successfully achieved all stated objectives. A fully functional multi-segment network topology was designed, implemented, and verified using Cisco 2901 routers and a Cisco switch. Both HTTP and HTTPS services were deployed on the server and confirmed accessible from both client workstations. Static routing, ACL filtering, and QoS prioritisation were configured and validated.

Empirical measurements confirmed that HTTP exhibits lower mean latency than HTTPS, with the additional overhead attributable to TLS handshake and encryption processing. QoS enforcement reduced latency variance, improving traffic predictability. Python automation scripts successfully collected router state via SSH and measured application-layer performance metrics without manual intervention. A real-time Flask dashboard provided an integrated visualisation of all collected data.

The project deepened the team's understanding of network protocol behaviour, Cisco IOS configuration, and modern Python-based automation approaches. The skills developed—troubleshooting connectivity, configuring security and QoS policies, and implementing programmatic network management—are directly applicable to contemporary network engineering roles.

Team Contribution

Table 4.1. Project Contribution Summary

Member	Primary Role	Key Contributions
Abd Alrahman Ismaik	Python Automation & Dashboard Developer	Netmiko SSH scripts; Scapy capture scripts; performance measurement scripts; Flask real-time dashboard; data collection and visualisation pipeline
Tarek Alhafez	Network Design & Configuration Lead	Network topology design; router configuration (IP addressing, static routing, ACLs, QoS); connectivity verification; report writing and analysis
Rashid Alzarooni	Testing, Traffic Analysis & Documentation	Wireshark/Scapy traffic analysis; HTTP/HTTPS functional testing; screenshot capture; report formatting and presentation preparation

All team members contributed to report writing and presentation preparation. Task allocation reflected individual strengths while ensuring cross-functional awareness of the complete system.